Análisis de Veyon

Virtual Eye On Networks — versión 4.11.4 (x86-64, Windows)

Informe completo de ingeniería inversa:
estructura, binarios, capacidades del sistema y desensamblado de los mecanismos centrales

Fuentes analizadas: instalación oficial de Windows (ZIP, 140 archivos) • código fuente github.com/veyon/veyon (tag 4.11.4) • repositorio de herramientas REA

Generado el 3 de noviembre de 2026

1. Resumen ejecutivo

Veyon es software **libre y de código abierto (GPLv2)** de **monitorización y control de ordenadores**, pensado para la gestión de aulas informáticas y soporte remoto. Permite a un profesor ver en miniatura, controlar, bloquear y difundir las pantallas de los equipos de los alumnos, enviar mensajes, lanzar programas, transferir archivos y apagar/encender equipos.

Este informe combina dos niveles de evidencia: el **análisis de los binarios reales** de la instalación de Windows (tablas de importación, exportaciones, cadenas y desensamblado x86-64) y la lectura del **código fuente de la misma versión 4.11.4**. Se comprobó que los símbolos y cadenas del código fuente aparecen literalmente dentro de los `.dll`/`.exe` instalados, por lo que las conclusiones corresponden exactamente a lo que ejecuta el paquete analizado.

Hallazgos clave

- **Arquitectura cliente-servidor** en C++/Qt 6, compilada con MinGW/GCC (x86-64), con criptografía vía QCA/OpenSSL.
- El paquete analizado es el **lado controlado (alumno) + configurador**: no incluye `veyon-master.exe` (consola del profesor).
- Mecanismo central en Windows: un **servicio SYSTEM** que **suplanta la sesión del usuario** (`CreateProcessAsUser`) para ejecutar su agente dentro del escritorio del alumno.
- Autenticación por **reto-respuesta asimétrico RSA-4096 / SHA-512**; la clave privada nunca viaja por la red.
- Diseño modular: cada función remota es un **plugin (DLL)**; toda la interacción con el sistema operativo se concentra en `windows-platform.dll`.
- Capacidades de **nivel kernel** (driver Interception para teclado/ratón) y una **API REST** embebida (webapi).
- Software legítimo y **firmado digitalmente** (cadena Certum); funcionalmente es una herramienta de control remoto potente que debe desplegarse solo con consentimiento de los usuarios y de la organización.

2. Metodología y alcance

El análisis es **estático** (no se ejecutó el software). Se emplearon las siguientes técnicas y herramientas:

- **Inventario** del paquete: descompresión del ZIP y clasificación de ejecutables, DLLs, plugins y recursos.
- **file / identificación PE**: tipo de binario, arquitectura y si están despojados de símbolos (stripped to external PDB).
- **objdump -x**: tablas de importación (APIs de Windows usadas) y de exportación (clases y métodos propios).
- **objdump -d**: desensamblado x86-64 de secciones de código.
- **strings**: extracción de cadenas (nombres de funciones C++ mangladas, mensajes, rutas, parámetros).
- **Lectura de código fuente** (github.com/veyon/veyon, 4.11.4) para interpretar con exactitud los mecanismos.
- **Correlación binario↔fuente**: verificación de que cada símbolo del código fuente existe en los binarios.

Archivos de origen

Origen	Detalle
Instalación Windows (ZIP)	140 archivos, ~61 MB descomprimidos; Veyon 4.11.4 x86-64 (MinGW)
Código fuente	github.com/veyon/veyon, tag v4.11.4 (C++/Qt, GPLv2)
Herramienta REA	github.com/morluto/rea — MCP de ingeniería inversa (referencia)

3. Qué es Veyon y qué funciones ofrece

Veyon = **Virtual Eye On Networks**. Copyright (c) 2004–2026 Tobias Junghans / Veyon Solutions. Licencia GNU GPL v2. Multiplataforma (Windows, Linux). Funciones principales declaradas:

Función	Descripción
Overview	Monitorizar todos los equipos de una o varias aulas en miniatura.
Remote access	Ver o controlar un equipo para asistir al usuario.
Demo	Difundir la pantalla del profesor en tiempo real (pantalla completa o ventana).
Screen lock	Bloquear pantalla/teclado/ratón para captar la atención.
Communication	Enviar mensajes de texto a los alumnos.
Lessons	Iniciar/cerrar sesión de todos los usuarios a la vez.
Screenshots	Registrar el progreso o documentar incidencias.
Programs & websites	Lanzar programas y abrir URLs remotamente.
Teaching material	Distribuir y abrir documentos, imágenes y vídeos.
Administration	Encender/apagar y reiniciar equipos remotamente.

4. Arquitectura general

Veyon se estructura en varios ejecutables que comparten un núcleo común (`veyon-core.dll`) y cargan funciones mediante plugins. El modelo es **profesor** ↔ **alumno** sobre el protocolo **RFB/VNC** con un tipo de seguridad propio.

Componente	Dónde corre	Función
`veyon-master*`	PC del profesor	Consola de control (NO incluida en este paquete).
`veyon-service`	Servicio del sistema	Arranca con el equipo, detecta la sesión activa y lanza server/worker en ella.
`veyon-server`	Sesión del alumno	Servidor VNC + autenticación + control de acceso; sirve pantalla y recibe órdenes.
`veyon-worker`	Sesión del alumno	Ejecuta las funciones (features) en el contexto del usuario.
`veyon-configurator`	GUI	Configuración: claves, red, control de acceso, LDAP.
`veyon-cli / wcli`	Línea de comandos	Administración y automatización.

* Componente del profesor, ausente en el paquete analizado (instalación de equipo controlado).

Puertos TCP por defecto

Puerto	Servicio
11100	veyon-server (pantalla VNC + control + autenticación)
11300	FeatureWorkerManager (comunicación con el worker)
11400	Demo server (difusión de la pantalla del profesor)

5. Inventario y análisis de binarios

5.1 Ejecutables (PE32+, x86-64, despojados a PDB externo)

Ejecutable	Tamaño	Tipo
`veyon-configurator.exe`	370.624 B	PE32+ GUI x86-64
`veyon-wcli.exe`	166.336 B	PE32+ GUI x86-64
`veyon-cli.exe`	165.824 B	PE32+ consola x86-64
`veyon-server.exe`	150.464 B	PE32+ GUI x86-64
`veyon-worker.exe`	55.232 B	PE32+ GUI x86-64
`veyon-service.exe`	27.584 B	PE32+ GUI x86-64
`uninstall.exe`	212.808 B	PE32 (32-bit) — desinstalador NSIS

Todos los ejecutables son envoltorios finos: su lógica real reside en `veyon-core.dll`. Compilados con MinGW (presencia de `libstdc++-6.dll`, `libwinpthread-1.dll`, `libgcc_s_seh-1.dll`).

5.2 Librerías principales

Librería	Rol
`veyon-core.dll`	Núcleo compartido: red, autenticación, features, servicio, cliente VNC.
`Qt6 Core/Gui/Widgets/Network/WebSockets/HttpServer`	Framework base (UI, red, JSON, servidor HTTP).
`libqca-qt6.dll + crypto/libqca-ossl.dll`	Qt Cryptographic Architecture sobre OpenSSL (RSA, firma, cifrado).
`libssl-3-x64 / libcrypto`	OpenSSL 3 (TLS y primitivas criptográficas).
`libvncserver.dll / libvncclient.dll / vnchooks.dll / ddengine64.dll`	Pila VNC: servidor, cliente, hooks y captura de pantalla.
`libldap / liblber / libsasl2 / libkldap-light`	Integración LDAP / Active Directory.
`interception.dll`	Driver de kernel para teclado/ratón (bloqueo e inyección).

Dependencias de `veyon-core.dll` (objdump -x)

KERNEL32.dll msvcrt.dll libgcc_s_seh-1.dll libstdc++-6.dll
Qt6Core.dll Qt6Gui.dll Qt6Network.dll Qt6Widgets.dll
libqca-qt6.dll libvncclient.dll

Observación relevante: el núcleo **no llama directamente a la API de Windows** (no importa ADVAPI32, USER32, etc.). Toda la interacción con el sistema operativo está aislada en el plugin de plataforma.

6. Plugins (DLLs cargados dinámicamente)

Cada capacidad es un plugin independiente. Hallazgo del análisis: **ningún plugin de función llama directamente a Win32**; todos dependen solo del núcleo y de Qt, y canalizan lo específico del sistema a través de `windows-platform.dll`. Lo que distingue a cada plugin es su librería adicional.

Plugin	Dependencia distintiva	Función
`windows-platform`	ADVAPI32, USER32, WTSAPI32, interception, NETAPI32...	Toda la interacción con Windows (ver sección 7).
`webapi`	Qt6HttpServer + QJsonDocument	Servidor HTTP/REST que responde JSON; control externo.
`demo`	libqca-qt6	Difusión cifrada de la pantalla del profesor (puerto 11400).
`authkeys`	libqca-qt6	Generar/importar/exportar el par de claves RSA-4096.
`ldap`	libldap, liblber, libkldap-light	Descubrimiento de equipos/usuarios en LDAP/AD.
`remoteaccess`	core + Qt	Ver o controlar un equipo concreto.
`screenlock`	core + Qt	Bloqueo de pantalla/entrada.
`powercontrol`	core + Qt	Encender (WOL) / apagar / reiniciar.
`usersessioncontrol`	core + Qt	Inicio/cierre de sesión masivo.
`screenshot / filetransfer / textmessage / desktopservices`	core + Qt	Capturas, transferencia de archivos, mensajes, abrir programas/URLs.
`builtin-ultravnc / external-vnc / headless-vnc`	libvncserver	Variantes del servidor VNC que sirven la pantalla.
`builtindirectory`	core + Qt	Directorio de equipos definido manualmente.

7. Capacidades reales sobre el sistema

La prueba definitiva de lo que el software puede hacer son las **funciones de la API de Windows que importa** `windows-platform.dll`. A continuación, agrupadas por capacidad (extraídas con `objdump -x`).

Control del servicio de Windows (ADVAPI32)

Instala, elimina, arranca, detiene y configura su propio servicio, y corre como servicio del sistema.

```
OpenSCManagerW  CreateServiceW  DeleteService  StartServiceW  ControlService
ChangeServiceConfigW  ChangeServiceConfig2W  QueryServiceStatus  SetServiceStatus
RegisterServiceCtrlHandlerExW  StartServiceCtrlDispatcherW
```

Ejecutar procesos como el usuario conectado (ADVAPI32 + WTSAPI32 + USERENV)

MECANISMO CENTRAL: el servicio SYSTEM obtiene el token de la sesion del alumno y lanza su agente dentro de ella. Detallado y desensamblado en la seccion 8.

```
WTSGetActiveConsoleSessionId  WTSEnumerateSessionsW  WTSQueryUserToken
WTSEnumerateProcessesW  LogonUserW  OpenProcessToken  DuplicateTokenEx
ImpersonateLoggedOnUser  RevertToSelf  AdjustTokenPrivileges  LookupPrivilegeValueW
CreateProcessAsUserW  CreateEnvironmentBlock
```

Inyección y bloqueo de teclado/ratón (USER32 + interception.dll)

Control remoto (mover raton y teclear en el equipo del alumno) y bloqueo de entrada mediante hooks y driver de kernel (infranqueable).

```
SendInput  keybd_event  MapVirtualKeyW  VkKeyScanW  ToAscii  GetAsyncKeyState
GetKeyState  SetWindowsHookExW  CallNextHookEx  UnhookWindowsHookEx
[interception.dll]  interception_create_context / get_filter / ...
```

Ventanas, monitores y escritorios (USER32)

Conoce la ventana/programa activo, cambia al escritorio seguro (login/UAC), dibuja la ventana de bloqueo/demo superpuesta y enumera monitores.

```
EnumWindows  FindWindowW  GetForegroundWindow  GetWindowTextW  IsWindowVisible
OpenInputDesktop  SetThreadDesktop  GetThreadDesktop  CloseDesktop
SetLayeredWindowAttributes  SetWindowPos  ShowWindow  RedrawWindow  EnableWindow
EnumDisplayDevicesW  QueryDisplayConfig  DisplayConfigGetDeviceInfo
```

Sesiones, apagado y dominio (WTSAPI32 / ADVAPI32 / NETAPI32 / Secur32)

Apagado/reinicio/logoff, consulta del controlador de dominio y grupos del usuario (control de acceso por grupo/LDAP), lectura/escritura del registro y del visor de eventos.

```
InitiateShutdownW  ExitWindowsEx                    (apagar / reiniciar / cerrar sesion)
DsGetDcNameW  NetGroupEnum  NetUserGetGroups  NetLocalGroupEnum  GetUserNameExW
RegCreateKeyExW  RegOpenKeyExW  RegSetValueExW      (registro de Windows)
RegisterEventSourceW  ReportEventW                  (registro de eventos)
```

Procesos y red (KERNEL32 + IPHLPAPI + SETUPAPI)

Enumera y gestiona procesos, crea tuberias para comunicarse con sus hijos, evita la suspension del equipo, obtiene MAC (Wake-on-LAN) e instala el driver Interception.

```
CreateToolhelp32Snapshot  Process32FirstW  Process32NextW  OpenProcess
ProcessIdToSessionId  TerminateProcess  CreatePipe  DuplicateHandle
SetThreadExecutionState   [IPHLPAPI: interfaces/MAC]   [SETUPAPI: instalar driver]
```

8. Mecanismo central #1: suplantación de sesión

Cómo el servicio SYSTEM ejecuta su agente dentro de la sesión interactiva del alumno. Implementado en `WindowsCoreFunctions::runProgramInSession` (compilado en `windows-platform.dll`). Secuencia exacta:

- **1. Auto-elevación de privilegios**: `enablePrivilege(SE_ASSIGNPRIMARYTOKEN_NAME, SE_INCREASE_QUOTA_NAME, SE_TCB_NAME)` vía `AdjustTokenPrivileges`. SE_TCB = "actuar como parte del SO".
- **2. Localizar proceso del usuario**: `findUserProcessId()` con WTS + `WTSGetActiveConsoleSessionId`.
- **3. Robar su token**: `OpenProcess(PROCESS_ALL_ACCESS)` → `OpenProcessToken(MAXIMUM_ALLOWED)`.
- **4. Reconstruir entorno**: `CreateEnvironmentBlock` + `SHGetKnownFolderPath(FOLDERID_Profile)`.
- **5. Suplantar al usuario**: `ImpersonateLoggedOnUser` (con `qScopeGuard(RevertToSelf)`).
- **6. Duplicar token como primario**: `DuplicateTokenEx(TOKEN_ASSIGN_PRIMARY, TokenPrimary)`.
- **7. Lanzar en el escritorio del alumno**: `CreateProcessAsUser(...)` con `si.lpDesktop = "Winsta0\\Default"`.

Código fuente (fragmento literal, WindowsCoreFunctions.cpp)

```
enablePrivilege( SE_ASSIGNPRIMARYTOKEN_NAME, true );
enablePrivilege( SE_INCREASE_QUOTA_NAME, true );
enablePrivilege( SE_TCB_NAME, true );

SmartHandle userProcessHandle{ OpenProcess(PROCESS_ALL_ACCESS, false, baseProcessId) };
OpenProcessToken(userProcessHandle.get(), MAXIMUM_ALLOWED, userProcessToken.put());
CreateEnvironmentBlock(userEnvironment.put(), userProcessToken.get(), false);
SHGetKnownFolderPath(FOLDERID_Profile, 0, userProcessToken.get(), profileDir.put());
ImpersonateLoggedOnUser(userProcessToken.get());
const auto revertToSelfGuard = qScopeGuard(RevertToSelf);

STARTUPINFO si{};  si.cb = sizeof(STARTUPINFO);
si.lpDesktop = desktopWide.get();            // "Winsta0\\Default"

DuplicateTokenEx(userProcessToken.get(), TOKEN_ASSIGN_PRIMARY|TOKEN_ALL_ACCESS,
                 nullptr, SecurityImpersonation, TokenPrimary, newToken.put());

CreateProcessAsUser(
        newToken.get(),              // token del alumno
        nullptr, commandLine.get(),  // "veyon-server.exe" ...
        nullptr, nullptr, heredaHandles,
        CREATE_UNICODE_ENVIRONMENT | NORMAL_PRIORITY_CLASS,
        fullEnvironment, profileDir.get(),
        &si, &pi );
```

Verificación en el binario (desensamblado x86-64)

La entrada de `CreateProcessAsUserW` está en la IAT de `windows-platform.dll` en el offset `0x639ba`. Prólogo real de una función del módulo (`objdump -d`):

```
Disassembly of section .text:
000000036eff1000 <.text>:
  36eff1000: 41 55              push   %r13
  36eff1002: 41 54              push   %r12
  36eff1004: 55                 push   %rbp
  36eff1005: 57                 push   %rdi
  36eff1006: 56                 push   %rsi
  36eff1007: 53                 push   %rbx
  36eff1008: 48 83 ec 28        sub    $0x28,%rsp
  36eff100c: 48 89 cf           mov    %rcx,%rdi        ; arg1 -> rdi
  36eff100f: 4d 89 c5           mov    %r8,%r13         ; arg3 -> r13
  36eff1012: 85 d2              test   %edx,%edx
  36eff1014: 0f 85 9e 00 00 00  jne    0x36eff10b8
```

El uso de la convención de llamada x64 de Windows (rcx, rdx, r8, r9) confirma que es un binario nativo Windows x86-64 genuino. Los nombres de las APIs de seguridad/WTS aparecen en la tabla de importación del mismo módulo (sección 7), lo que corrobora el flujo anterior.

9. Mecanismo central #2: autenticación por firma

La conexión profesor→alumno viaja sobre RFB/VNC con un tipo de seguridad propio de Veyon (`rfbSecTypeVeyon = 40`). El modo recomendado es **KeyFile**: un reto-respuesta asimétrico en el que el profesor demuestra poseer una clave privada sin revelarla.

Parámetros criptográficos (CryptoCore.h, verificados en veyon-core.dll)

Parámetro	Valor
Tamaño de clave RSA	`RsaKeySize = 4096` bits
Tamaño del reto	`ChallengeSize = 128` bytes aleatorios
Algoritmo de firma	`EMSA3_SHA512` (RSA PKCS#1 v1.5 + SHA-512)
Algoritmo de cifrado (Logon)	`EME_PKCS1_OAEP`

Flujo del handshake (modo KeyFile)

Paso	Servidor (alumno)	Cliente (profesor)
1	`generateChallenge()` → envía 128 bytes aleatorios	—
2	—	Comprueba tamaño; firma: `key.signMessage(challenge, EMSA3_SHA512)`
3	—	Envía { nombreClave, firma }
4	Valida nombre; comprueba firma ≤ 8192 B (anti-DoS); carga clave pública; `verifyMessage(...)`	—
5	Si verifica → `rfbVncAuthOK` → control de acceso (`AccessControlProvider`)	—

Cliente — firma del reto (VeyonConnection.cpp)

```
const auto challenge = challengeReceiveMessage.read().toByteArray();
if( challenge.size() != CryptoCore::ChallengeSize ) return FALSE;   // 128

auto key = connection->authenticationCredentials().privateKey();
if( key.isNull() || key.canSign() == false ) return FALSE;

const auto signature =
        key.signMessage( challenge, CryptoCore::DefaultSignatureAlgorithm );  // EMSA3_SHA512

challengeResponseMessage.write( ...authenticationKeyName() );
challengeResponseMessage.write( signature );
challengeResponseMessage.send();
```

Servidor — verificación (ServerAuthenticationManager.cpp)

```
case VncServerClient::AuthState::Init:
    client->setChallenge( CryptoCore::generateChallenge() );   // 128 bytes
    ...write( client->challenge() ).send();
    return AuthState::Challenge;

case VncServerClient::AuthState::Challenge:
    const auto authKeyName = message.read().toString();
    const auto signature   = message.read().toByteArray();

    constexpr int MaxSignatureSize = 8192;          // RSA-4096 = 512 B + margen
    if (signature.size() > MaxSignatureSize) return AuthState::Failed;

    const auto publicKeyPath = VeyonCore::filesystem().publicKeyPath( authKeyName );
    CryptoCore::PublicKey publicKey( publicKeyPath );
    if( publicKey.isNull() || publicKey.isPublic() == false ) return AuthState::Failed;

    if( publicKey.verifyMessage( client->challenge(), signature,
                                 CryptoCore::DefaultSignatureAlgorithm ) == false )
        return AuthState::Failed;
    return AuthState::Successful;
```

Correlación binario ↔ fuente (símbolos hallados en los binarios)

```
veyon-core.dll :
  _ZN10CryptoCore17generateChallengeEv
  _ZN3QCA10PrivateKey11signMessageE...SignatureAlgorithmE...SignatureFormatE
  _ZN3QCA10PrivateKey7decryptE...EncryptionAlgorithmE

veyon-server.exe :
  ServerAuthenticationManager::performKeyAuthentication(VncServerClient*, VariantArrayMessage&)
  _ZN3QCA9PublicKey13verifyMessageE...  _ZNK10Filesystem13publicKeyPathERK7QString
  _ZN21AccessControlProvider11checkAccessE...
  cadenas: "signature size exceeds maximum of"  "invalid auth key name!"
           "failed to load public key from"     "failed to send challenge"
```

Los otros dos modos: **Logon** (el servidor envía su clave pública; el cliente cifra la contraseña del dominio con OAEP y la envía; el servidor la descifra y valida con `LogonUserW`) y **Token** (secreto compartido de 128 bytes comparado directamente).

10. Valoración de seguridad y legitimidad

- **Software legítimo y de código abierto** (GPLv2). No se observó comportamiento malicioso, ofuscación ni telemetría encubierta.
- **Binarios firmados** digitalmente (cadena Certum; aparece el endpoint OCSP `ccsca2021.ocsp-certum.com`), coherente con una build oficial.
- **Criptografía sólida**: RSA-4096, SHA-512, reto aleatorio por sesión, límite anti-DoS en el tamaño de firma; la clave privada nunca viaja.
- **Buenas prácticas de código**: punteros inteligentes (SmartHandle/SmartToken), `qScopeGuard(RevertToSelf)`, validación de tamaños, control de acceso por reglas/grupos.
- **Advertencia funcional**: pese a ser legítimo, es una herramienta de monitorización y control remoto con capacidades de nivel kernel (driver de entrada) y suplantación de sesión. Debe instalarse únicamente con autorización y con conocimiento de los usuarios y de la organización.
- **Superficie de ataque** a vigilar en un despliegue: puertos 11100/11300/11400, el driver Interception, la API REST (webapi) y la gestión/custodia de las claves públicas instaladas en cada equipo.

11. Conclusión

El paquete analizado es **Veyon 4.11.4 para Windows x86-64** (lado de equipo controlado + configurador). Binario a binario, es una suite de administración/vigilancia de aulas que: (1) corre como servicio SYSTEM y **suplanta la sesión del usuario** mediante `CreateProcessAsUser` para plantar su agente; (2) autentica al profesor con un **reto-respuesta RSA-4096 / SHA-512**; (3) sobre VNC permite ver, controlar, bloquear (con driver de kernel), difundir pantallas, capturar, transferir archivos y apagar/reiniciar/cerrar sesiones; y (4) se integra con dominio/LDAP y expone una API REST. El diseño es limpio, modular y técnicamente correcto; su alcance exige un despliegue responsable y consentido.

Informe de análisis estático con fines educativos y de evaluación. Veyon es marca y obra de Veyon Solutions / Tobias Junghans, distribuido bajo GNU GPL v2. Todo el análisis se realizó sobre material facilitado por el usuario y sobre el código fuente público oficial.